

Задание #1

- Зайти на Gitlab кафедры ВС. Далее будет подразумеваться именно этот Gitlab – <https://git.csc.sibsutis.ru>
- Создать репозиторий. Дальнейшая разработка будет вестись в этом репозитории.
- Сконфигурировать клиент git для использования имени и адреса email из учетной записи gitlab.
- Написать простое приложение, которое запускает http-сервер и отдает контент. Контент может быть любым – строка, текст, случайное число и тп. Язык программирования – любой.
- Написать Dockerfile для данного приложения. Результат сборки – docker image, при запуске docker-контейнера из которого, в контейнере запускается разработанное приложение. Файл разместить в репозитории проекта.

Задание #2

- Доработать Dockerfile – Для компилируемых языков сделать multistage сборку. В случае использования интерпретируемых ЯП, например, python – создать в отдельной папке Dockerfile и использовать программу «Hello World» на компилируемом языке.
 - а. Этап 1 – компиляция/сборка. Используется образ с наличием компилятора (Например FROM gcc)
 - б. Этап 2 – копирование артефакта компиляции и запуск приложения. Используется образ ОС (Например FROM alpine)
 - с. Проанализировать как изменится размер итогового образа.
 - Разработать конфигурацию Gitlab CI со следующим функционалом
 - а. Сборка образа (Build, tag)
 - б. Загрузка образа в Gitlab Registry (push)
 - с. Удаление образа с Gitlab Runner-a
- Допускается опираться на template ci, предлагаемый gitlab.
- Запуск CI должен выполняться автоматически при установке тега для коммита (git tag). Так же для любого коммита должна быть возможность запуска «по кнопке» (manual). Тегом (docker tag) получившегося docker-образа является хэш коммита или git tag при наличии.
 - Добавить в проект .dockerignore файл, включив в него список файлов, не требуемых для сборки проекта (Синтаксис аналогичен файлу .gitignore) или являющимися результатом компиляции. Проанализировать как изменится размер итогового образа и время сборки в зависимости от размера проекта (Для измерения времени и размера можно создать временный большой файл и добавить/убрать его из .dockerignore).
 - **Временный файл добавлять в индекс репозитория и коммитить не нужно.**
 - **При наличии программы Hello World собирать образ этой программы средствами CI не требуется. Сборка только основного проекта**

Задание #3

- Разработать `docker-compose.yml` файл со следующим функционалом:
 - a. Запуск образа приложения, собранного через CI в задании #2
 - b. Запуск готового docker-приложения с http-сервером (например – `traefik/whoami`)
 - c. Запуск прокси-сервера. Прокси-сервер должен терминировать ssl реализовать отдачу контента двух приложений при обращении к домену по протоколу `https`. Домены допускается использовать любые. Далее в качестве примера используется `domain1` и `domain2`
 - d. При обращении к `https://domain1` запрос должен быть проксирован приложению из пункта а. При обращении к `https://domain2` должно ответить приложение из пункта b. Варианты прокси-серверов (Включая, но не ограничивая):
 1. Nginx
 2. Traefik
 - e. Коммуникация между прокси-сервером и приложениями должна проходить в пределах вирт.сети `docker`, созданной `docker compose`
- `docker-compose.yml` файл, дополнительные файлы конфигурации разместить так же в репозитории. Файлы конфигурации поместить в отдельной директории.
- Сервисы, описанные в `docker-compose.yml` не должны иметь директиву `build`. Запуск производится из уже собранных/готовых образов задания #2. Для того, чтобы использовать образы из `registry` необходимо выполнить команду

`docker login registry.csc.sibsutis.ru`

- Для работы на ПК кафедры необходимо использовать домены вида **`*.local.csc.sibsutis.ru`** (прим. - `test.local.csc.sibsutis.ru`). Любой из поддоменов разрешится в адрес **`127.0.0.1`**

Задание #4

- Добавить в Docker-compose экземпляр базы данных. Примеры СУБД (Включая, но не ограничивая)
 - a. Mariadb
 - b. Postgresql
 - c. Mongoddb
- Доработать приложение для работы с базой данных.
 - a. Необходимо разработать модель данных (Например, User с полями id, name, email)
 - b. При запуске приложения должны создаваться необходимые для этой модели таблицы в базе данных.
 - c. Реализовать в приложении набор следующих endpoint-ов (На примере /users)
 - 1. GET /users – вывести список всех пользователей из БД (можно реализовать с поддержкой параметров start-end или иным вариантом пагинации, e.g.: /users?_start=0&_end=5)
 - 2. PUT /users – создать пользователя с переданными параметрами
 - 3. GET /users/{:id} – вывести данные пользователя с указанным id
 - 4. DELETE /users/{:id} – удалить пользователя с указанным id
 - d. Данные, передаваемые для put, post, delete оформляются в json, возвращаемые данные также в json.
 - e. Продемонстрировать взаимодействие с БД
- Параметры подключения к базе данных необходимо передавать приложению при запуске через ключи командной строки или переменные окружения (Environment)
- Для выполнения миграции схемы бд допускается использовать:
 - a. Запуск команды в entropoint-скрипте выполняющая миграцию (В качестве приложений, управляющих версией бд, можно использовать flyway, goose, etc...)
 - b. Решение, реализованное непосредственно в orm или библиотеке работы с бд
- Дополнительные файлы конфигурации разместить в отдельной директории.